LymphTrack

Technical Manual

Prepared by:
SIMON Pimprenelle

Affiliation:
Microwave in Medical Engineering Group (MMG)
Uppsala University

UPPSALA
UNIVERSITET

Document version : *1.0*

Table of Contents

1. Introduction 4

1.1 Purpose of the Application 4

1.2 Core Technologies 4

1.3 What This Manual Contains 5

2. Project Architecture 5

2.1 Overview 5

2.2 Data Directory Structure (C:\LymphTrackData) 6

2.3 Application Directory Structure (C:\LymphTrackApp) 7

2.4 Backend (FastAPI – Python) 7

2.5 Frontend (React / TypeScript / Expo) 7

3. Prerequisites and installation 8

3.1 Accessing the remote server 8

3.2 Required Tools 9

3.3 Completing the installation 10

3.4 Other project requirements 10

4. Starting the Backend 11

4.1 Manual start (development / debugging) 11

4.2 Automatic start (recommended) 12

5. Starting the frontend 13

5.1 Manual start (development / debugging) 13

5.2 Automatic start (recommended) 15

6. Database 15

6.1 Database Schema Diagram 15

6.2 Table overview 16

6.2.1 sick_patients 16

6.2.2 operations 16

6.2.3 results 16

6.2.4 photos 16

6.2.5 healthy_metadata 17

6.2.6 healthy_patients 17
6.2.7 users (managed by Supabase) 17
6.3 Authorization model and user privileges 17
6.3.1 Role Definitions 17
6.3.2 Authorization logic in the backend 18
6.3.3 Why does it matters ? 18
6.4 Database Connection Testing 19
7. User guide 20
7.1 Accessing the application 20
7.2 Patients page 21
7.3 Creating a new patient 22
7.4 Exporting patient data 24
7.5 Viewing a patient profile 24
7.6 Creating a follow-up visit 26
7.7 Viewing a patient visit 27
7.8 Position page 29
7.9 Settings 30
7.9.1 Administrator – User management 31
7.9.2 Administrator – User creation and update 32
7.10 Outcomes 33
8. Future improvements and known issues 36
8.1 Future improvements 36
8.2 Known issues 38
9. Annex 39
9.1 Annex 1: LymphTrackApp directory structure 39
9.2 Annex 2: Database schema diagram 41
9.3 Annex 3: SQL schema 42

1. Introduction

1.1 Purpose of the Application

LymphTrack is a technical platform designed to visualize, track, and analyze the progression of lymphedema using microwave-based measurements, specifically S11 return-loss curves collected at multiple limb positions.

Its primary objectives are:

- Visualizing the evolution of lymphedema across visits, patients, and measurement positions
- Managing, storing, and processing Excel files containing per-patient, per-visit, and per-position measurements
- Providing an interactive interface to display S11 curves, compare measurements, and explore trends
- Facilitating multi-patient comparison, allowing clinicians to visually compare several individuals on the same graph

Taken together, the application serves as a unified tool for researchers to monitor patient progression, centralize raw measurement data, and access meaningful analytics in a simple and consistent interface.

1.2 Core Technologies

The application is built using a modular architecture relying on widely adopted frameworks and tools:

- **Backend:** FastAPI[1] (Python), providing API endpoints, data processing logic and file upload handling
- **Database and Authentication:** Supabase[2] (hosted PostgreSQL), serving as the central storage for users, patients, operations, and results, with managed authentication, password hashing, and token handling
- **Frontend:** React with Recharts for S11 curve visualization
- **File Storage:** Excel files and images are stored directly on the server's local storage
- **Deployment Environment**:
    - Backend running on a remote Windows-based university server
    - Frontend deployed on the same remote, configured to communicate directly with the backend through local HTTP.
- **Testing and API Validation:** pytest and requests

[1] FastAPI is a modern, high-performance Python web framework for building APIs.
[2] Supabase is an open-source backend platform that provides authentication, storage, and a managed PostgreSQL database.

This technology stack provides a lightweight, scalable, and maintainable foundation suitable for both development and production deployments.

1.3 What This Manual Contains

This technical manual provides all the necessary information to understand the structure, deployment process, and usage of LymphTrack. It includes:

- A complete explanation of the project architecture (backend, frontend, database, file storage)
- Detailed steps for installation and configuration on a remote machine
- Instructions for starting and running both backend and frontend services
- Documentation on database structure
- A user tutorial describing how to log in, upload data, navigate graphs, and use comparative visualizations

This manual is intended to give developers, clinicians, and technical maintainers a complete reference to understand, deploy, maintain, and use LymphTrack efficiently.

2. Project Architecture

This section presents a complete overview of the architecture of LymphTrack. The project is composed of two complementary components:

1. **The application layer** (backend, frontend, scripts)
2. **The data storage layer** containing all raw measurement files

Both components are required for the system to operate correctly.

2.1 Overview

LymphTrack is divided into two main directories located at the root of the remote server:

1. Application (C:\LymphTrackApp)

This directory contains the full source code and execution environment of the platform:

- The **backend** (FastAPI + Python) exposes the API endpoints, processes measurement files and inserts parsed results into the database.
- The **frontend** (React / TypeScript / Expo Router) provides the graphical interface used by clinicians and researchers.

2. Storage (C:\LymphTrackData)

This directory contains **all raw measurement data** used by the backend:

- One folder per patient
- Subfolders per visit
- The original Excel files produced during microwave measurements
- Additional files such as operation-related photos and generated plot

Together, **C:\LymphTrackApp** and **C:\LymphTrackData** form the complete runtime environment. The first provides the application logic, while the second provides all input data required by the backend.

Below is the full structure of both folders.

2.2 Data Directory Structure (C:\LymphTrackData)

```
C:\LymphTrackData
|   // Root directory storing all patient measurement data
|
+---MV001/                      // Patient folder (ID = MV001)
|   |   .DS_Store               // System file (macOS artifact)
|   |
|   +---1-PreOP_13052018/       // Visit folder (name + date)
|   |   |   .DS_Store
|   |   |
|   |   +---1/                          // Position 1 measurements
|   |   |       combined_plot.png       // Pre-generated plot
|   |   |       VNA_180513_161055_1.xls // Raw S11 measurement files
|   |   |       VNA_180513_161108_2.xls
|   |   |       VNA_180513_161121_3.xls
|   |   |
|   |   +---2/
|   |   |       combined_plot.png
|   |   |       VNA_180513_161209_4.xls
|   |   |       VNA_180513_161219_5.xls
|   |   |       VNA_180513_161231_6.xls
|   |   |
|   |   +---3/
|   |   +---4/
|   |   +---5/
|   |   \---6/
|   |
|   \---2-1_year_25052019/
|
+---MV002/
+---MV003/
|
+---…
|
\---MV130/
```

Figure 2.2: Excerpt of the data directory structure

2.3 Application Directory Structure (C:\LymphTrackApp)

The detailed folder structure (C:\LymphTrackApp) can be found in the annexes (see **Annex 1**).

2.4 Backend (FastAPI – Python)

The backend contains all logic related to data ingestion, processing, and API exposure.

Key responsibilities

- Providing REST API endpoints (patients, operations, results, photos…)
- Parsing measurement files (Excel, CSV)
- Computing microwave features (min S11, resonance frequency, bandwidth)
- Managing database access using an ORM[3]
- Handling authentication and user access rules

Directory summary

- app/main.py: Entry point of the FastAPI application
- app/api/: All API route definitions (CRUD[4] operations used by the frontend)
- app/db/: Database engine, ORM models
- app/core/: Application settings and configuration
- app/auth/: Authentication utilities (token verification, user permissions, etc.)
- app/tests/: Automated tests (pytest)

Running the backend

Startup instructions are provided in the dedicated section: **4. Starting the backend**

2.5 Frontend (React / TypeScript / Expo)

The frontend is a React application using Expo Router for navigation. It allows clinicians and researchers to explore patient data, visualize S11 curves, and interact with follow-up visits.

Key responsibilities

- User Interface for managing patients and visits
- Dynamic graph visualisation of microwave S11 curves
- File uploads and form-based data entry
- Multi-patient comparison views
- User authentication
- Access to settings and administrative tools

[3] ORM (Object-Relational Mapping) is a technique that maps database tables to programming language classes, allowing data to be handled as objects instead of SQL queries.
[4] CRUD refers to the four basic operations on data: Create, Read, Update, and Delete.

Directory summary

- app/: All screens and routed pages
- components/: Reusable UI components
- constants/: Colors, styles, API URL
- utils/: Helper functions
- assets/: Logos, icons, images
- lib/: Supabase client (authentication)
- dist/: Compiled frontend (web version) used for deployment

Running the frontend

Startup instructions are provided in the dedicated section: **5. Starting the frontend**

3. Prerequisites and installation

This section describes all requirements needed to configure and run LymphTrack on the remote university server.

Since the backend, the frontend, data files, and operational environment run exclusively on the remote Windows machine, **no local installation is required for normal usage**.

3.1 Accessing the remote server

This section describes how to install and configure the project on the university's Windows server.

Access to the remote Windows server is done through **Remote Desktop** using the university's network.

Steps to connect

1. On your computer, open the application **Remote Desktop Connection** *(search "Remote Desktop" in the Windows start menu)*
2. In the "Computer" field, enter the server address:

```
mmg-rdp.angstrom.uu.se
```

3. Click **Connect**.
4. Enter the username and password provided by Uppsala University.

5. You must be connected to:
    - the **Eduroam** WiFi network, **or**
    - the **Uppsala University VPN** (required if working from home or externally)

Important note

All code and data used by LymphTrack are stored in two directories on the server:

```
C:\LymphTrackApp      ← application source code (backend + frontend)
C:\LymphTrackData     ← measurement files and patient-related data
```

3.2 Required Tools

LymphTrack operates entirely on the remote Windows Server provided by Uppsala University. This is the only environment that must meet the required specifications.

Preinstalled on the server:

- **Node.js:** Required to run the React frontend (Expo Router). Verify the installation using the following commands in PowerShell:

```
node -v
npm -v
```

- **Git:** Used to clone and update the repository. To verify:

```
git --version
```

- **Python 3.13:** Used for the FastAPI backend and all backend scripts. To verify:

```
python --version
```

Optional but recommended:

- **Visual Studio Code:** Must be installed individually by each user because administrative permissions are restricted. VS Code is useful for maintenance, debugging, or editing the backend and frontend source code.

Download at https://code.visualstudio.com/

After the installation, open VS Code go on *File* and click on *Open Folder* then select the following folder:

```
C:\LymphTrackApp
```

3.3 Completing the installation

Once you are connected to the server, dependencies must be installed for the backend and frontend.

Backend dependencies:

```
cd C:\LymphrTrackApp\backend

.\venv\Scripts\Activate.ps1

pip install -r requirements.txt
```

Frontend dependencies:

The React/Expo frontend includes its own dependency folder (node_modules or .expo). If dependencies need to be reinstalled:

```
cd C:\LymphTrackApp\frontend

npm install
```

3.4 Other project requirements

In addition to the software installed on the server, LymphTrack relies on several external cloud services and development platforms. These tools are not installed locally on the server but are required for maintaining, updating or redeploying the application.

Supabase

Supabase provides the backend infrastructure used by LymphTrack. It is used for:

- PostgreSQL **database** hosting
- **Authentication** and session management
- **User management**
- **Role-based permissions** through policies

Credentials are stored securely as environment variables in the backend (**.env**). Anyone maintaining or extending the project will need access to the Supabase workspace to inspect the database schema or update security rules.

GitHub

GitHub contains the **source code** of the project for both backend and frontend. It is required for:

- version control
- code updates and collaborative development
- maintaining documentation

Access to the repository is necessary for future contributors wishing to update the application.

4. Starting the Backend

The backend can be started in two ways, depending on the intended use:

- **Manual mode**: for debugging, maintenance, or editing the backend directly.
- **Automatic mode (recommended)**: starts both backend and frontend together through a single script.

4.1 Manual start (development / debugging)

This mode is used when working directly on the backend code.

1. **Open a terminal**

Either PowerShell or the VSCode integrated terminal.

2. **Navigate to the backend folder**

```
cd C:\LymphTrackApp\backend
```

3. **Activate the Python virtual environment**

```
.\venv\Scripts\Activate.ps1
```

4. Start FastAPI

```
uvicorn app.main:app --host 0.0.0.0 --port 8000
```

5. Verify the backend is running

Open a browser on the remote server itself:

```
http://localhost:8000/docs
```

If Swagger UI appears, the backend is running correctly. It should look similar to the screenshot below.

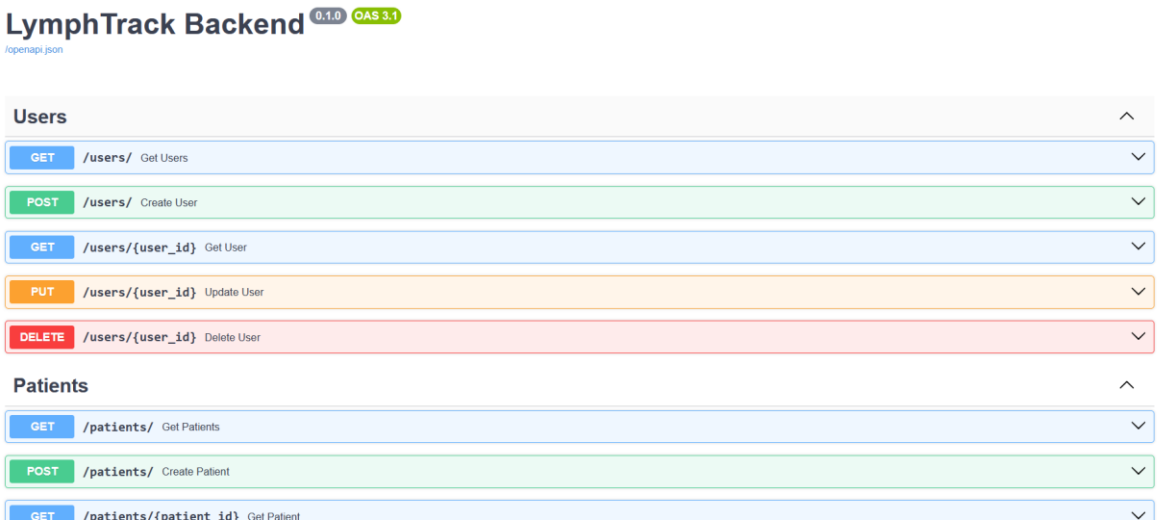


Figure 4.1: Swagger UI displayed when the backend is running correctly

4.2 Automatic start (recommended)

For normal use, the backend is started through the global script located at the root of the project:

```
C:\LymphTrackApp\start_app.ps1
```

Running this script:

- builds the frontend,
- starts FastAPI on port 8000,
- serves the frontend on port 3000,

- and opens the application in a browser.

To start the full application:

```
cd C:\LymphTrackApp

.\start_app.ps1
```

When the script runs, two PowerShell windows open in the background. These windows correspond to the backend and the frontend servers.

They must remain open for the application to function.

This is the recommended method for daily usage, demonstrations, and testing.

5. Starting the frontend

The frontend is built with React Native and Expo, and is served directly from the remote machine. It can be started in two ways:

- **Manual development mode**: used when editing or debugging the frontend.
- **Automatic mode (recommended)**: started together with the backend through the global script.

This section explains how to start the frontend in both contexts and how to verify that it operates correctly.

5.1 Manual start (development / debugging)

This mode is used when modifying or testing the frontend code.

1. **Open a terminal**

Either PowerShell or the VSCode integrated terminal.

2. **Navigate to the frontend directory**

```
cd C:\LymphTrackApp\frontend
```

3. Install dependencies if not already (first time only)

```
npm install
```

4. Set the API URL

In:

```
frontend/constants/api.ts
```

Use:

```
export const API_URL = "http://130.238.147.150:8000";
```

5. Start the development server

```
npx expo start
```

Press **w** to open the app in a browser. This mode is intended only for development.

6. Verify that the UI loads

Once the frontend starts, the login screen should appear.

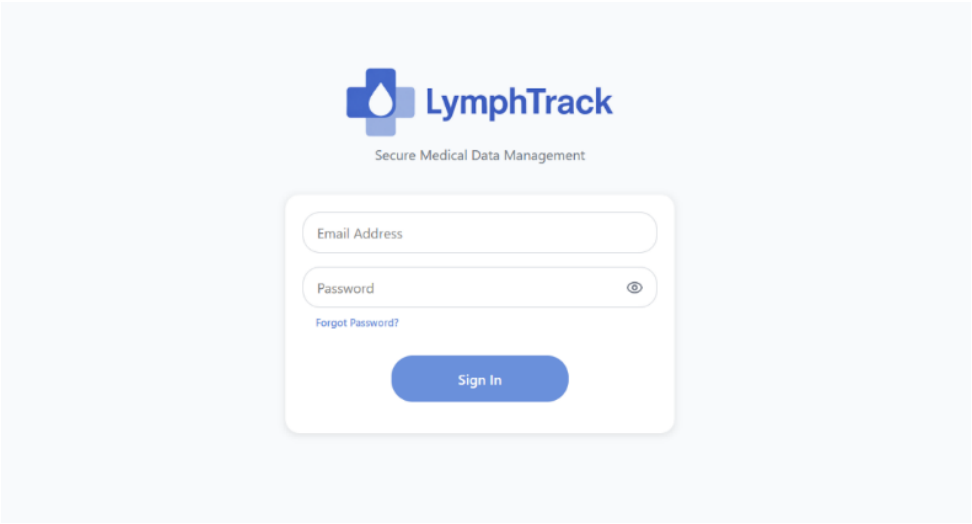


Figure 5.1: Frontend login screen

5.2 Automatic start (recommended)

The frontend does not need to be started manually. It is automatically built and served when launching the application through the global startup script.

For instructions, see **4.2 Automatic start (recommended)**

which describes how to start the full application using:

```
C:\LymphTrackApp\start_app.ps1
```

This script builds the frontend, starts the backend, serves the web application, and opens it in the browser.

6. Database

LymphTrack uses a Supabase-hosted **PostgreSQL**[5] database for persistent storage of patients, operations, S11 measurement results, healthy reference data, and user accounts.

This section describes the schema structure, table relationships, and the role of each component in the system.

6.1 Database Schema Diagram

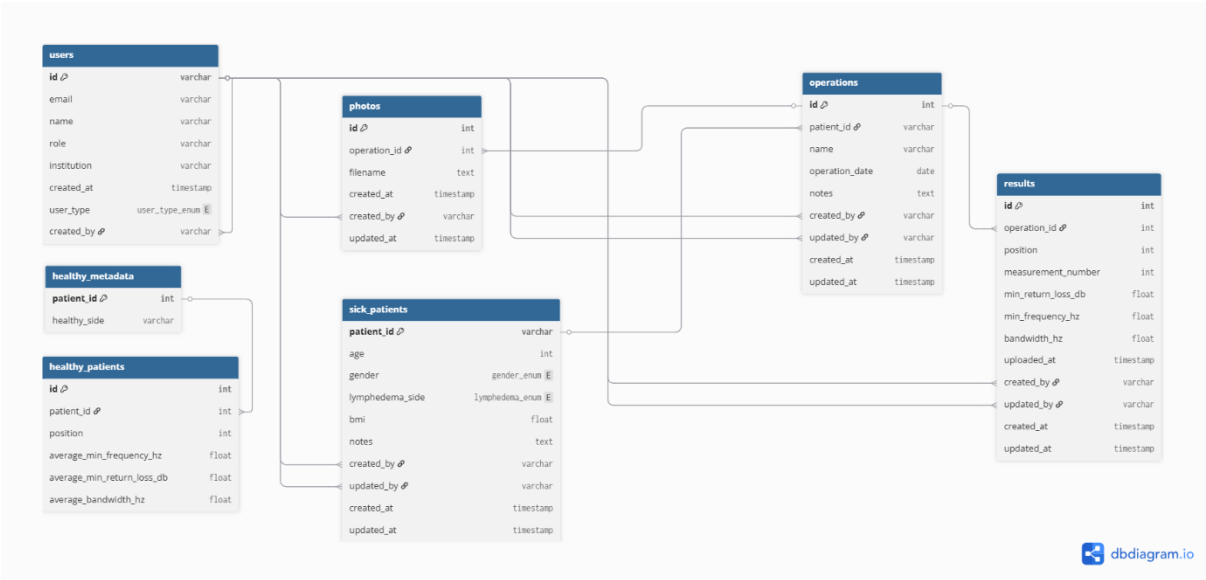


Figure 6.1 : Database schema diagram

[5] PostgreSQL is an open-source, reliable relational database system.

Figure 6.1 shows the complete relational schema of the LymphTrack database. Due to size constraints, a high-resolution version of the diagram is provided in **Annex 2.**

The schema is organized around three main domains:

- Patient management (sick and healthy patients)
- Clinical operations and microwave measurement results
- User accounts, roles and authentication (Supabase)

6.2 Table overview

The complete SQL schema is included in **Annex 3**, and is also available in the project directory under: **C:\LymphTrackApp\database\schema.sql**

The following subsections describe the purpose and role of each table in the system.

6.2.1 sick_patients

Stores demographic and clinical metadata for lymphedema patients, including age, gender, BMI, lymphedema side, and optional notes.Each patient is uniquely identified by a patient_id.

This table represents the core clinical population processed by the application.

6.2.2 operations

Represents each clinical visit or measurement session linked to a patient (e.g., pre-operative visit, 1-year follow-up). An operation can contain multiple S11 microwave measurements and optional clinical photos.

This table structures longitudinal follow-up for each patient.

6.2.3 results

Contains the processed microwave features extracted from raw VNA measurements, including minimum S11, resonance frequency, and bandwidth.

These values are used for graph visualisation, comparison between visits, and automated analysis modules.

6.2.4 photos

Stores filenames of optional clinical images associated with a given operation.

Each photo is linked to a specific operation and uploaded through the application.

6.2.5 healthy_metadata

Contains metadata describing the healthy reference side for each healthy volunteer.

Although not currently used directly by the application, this table was included in the database to support **future extensions**, such as automatic comparison between healthy and affected sides in asymmetrical analysis.

6.2.6 healthy_patients

Stores averaged microwave features measured on healthy volunteers for each anatomical position.

These values may be used in the future to compute healthy–affected asymmetry ratios or for threshold-based diagnostic models.

6.2.7 users (managed by Supabase)

Managed automatically by Supabase Authentication.

Stores application users, their email, role, institution, and creation metadata.The user_type field defines role-based permissions (user, admin, super_admin), which determine access and modification rights in the system.

6.3 Authorization model and user privileges

LymphTrack implements a role-based authorization system built on top of Supabase Authentication, PostgreSQL Row-Level Security (RLS), and backend-side permission checks.

This ensures that sensitive clinical data can only be viewed or modified by authorized users.

Three user roles exist in the system:

- **super_admin**
- **admin**
- **user**

These roles are stored in the users table via the user_type ENUM.

6.3.1 Role Definitions

user_type	Permissions
user	• Can view patient files • Can view operations, results, photos

	• Can export patient data and graphs • Cannot create, edit, or delete anything • Cannot manage user accounts

admin	• All user permissions
	• Can **create**, **edit**, and **delete**:
	  ○ patients
	  ○ operations
	  ○ photos
	  ○ results
	• Can manage (create/edit/delete) **standard users**
	• Cannot modify or delete **other admins**
	• Cannot create or edit **super_admin** accounts

super_admin	• All admin permissions
	• Can create/edit/delete:
	  ○ **users**
	  ○ **admins**
	  ○ **other super_admins**

6.3.2 Authorization logic in the backend

Even though Supabase storage and tables enforce access rules through SQL policies, **the backend adds an extra security layer** before performing any write operation.

Before any update/delete, the backend checks:

1. Current user role
2. Target user or resource role
3. Allowed privilege level

This prevents privilege escalation or accidental modifications.

6.3.3 Why does it matters ?

This hierarchical system protects clinical and research data by ensuring:

- No standard user can alter patient data
- Only trusted admins can perform write operations
- Only project owners can manage privileged accounts
- No administrator can accidentally demote or delete themselves
- Database integrity is preserved across environments

This model also supports future extensions such as audit logs, per-hospital permissions, or temporary elevated privileges.

6.4 Database Connection Testing

To verify that the backend can successfully connect to the Supabase PostgreSQL database, a dedicated test script is included in the project:

C:\LymphTrackApp\backend\app\tests\db\test_db.py

This script performs a direct connection attempt using the DATABASE_URL defined in the environment variables. It bypasses FastAPI entirely, making it a simple and reliable way to diagnose connectivity issues.

Purpose

This script is used to:

- Validate that the DATABASE_URL is correctly defined
- Confirm that the remote Supabase PostgreSQL instance is reachable
- Detect SSL misconfigurations
- Ensure that backend environment variables are loaded correctly
- Diagnose connection issues independently from FastAPI

This makes it extremely useful when:

- Setting up the backend for the first time
- Configuring a new environment (local or remote)
- Running diagnostics if the API cannot access the database

How to run the test

1. **Navigate to the backend directory**

```
cd C:\LymphTrackApp\backend
```

2. **Activate the virtual environment**

```
.\venv\Scripts\Activate.ps1
```

3. **Run the script**

```
python app/tests/db/test_db.py
```

Expected output

If everything is configured correctly:

```
Connection successful to Supabase PostgreSQL!
```

If something is wrong (URL, password, networking, SSL, etc.):

```
Connection error: <error message>
```

This allows quick troubleshooting without running the full FastAPI backend.

7. User guide

This section explains how to use the LymphTrack application, from accessing the web interface to navigating patient data, importing measurements, visualizing S11 curves and exporting visit information.

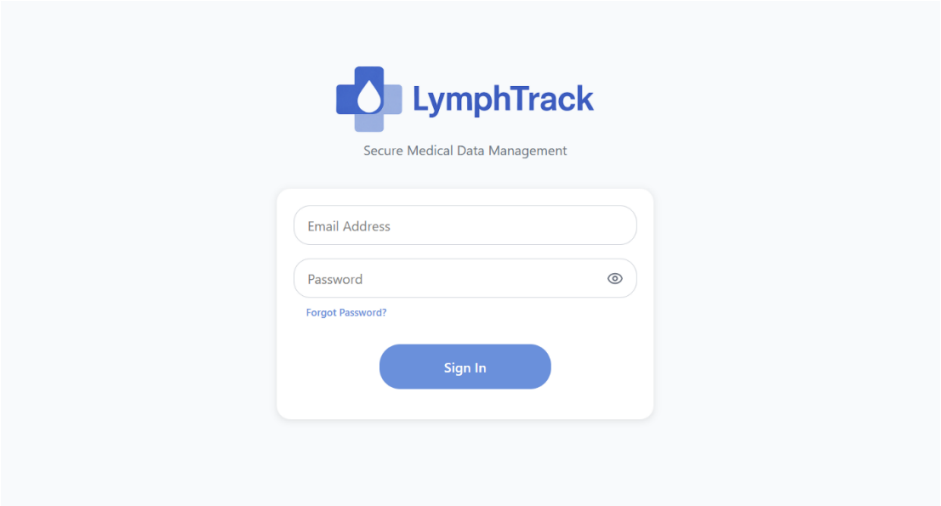
Access to the platform is restricted to authorized medical personnel. Users can sign in with the credentials provided by the research group.

7.1 Accessing the application

To access the LymphTrack interface, open your browser and navigate to:

http://130.238.147.150:3000

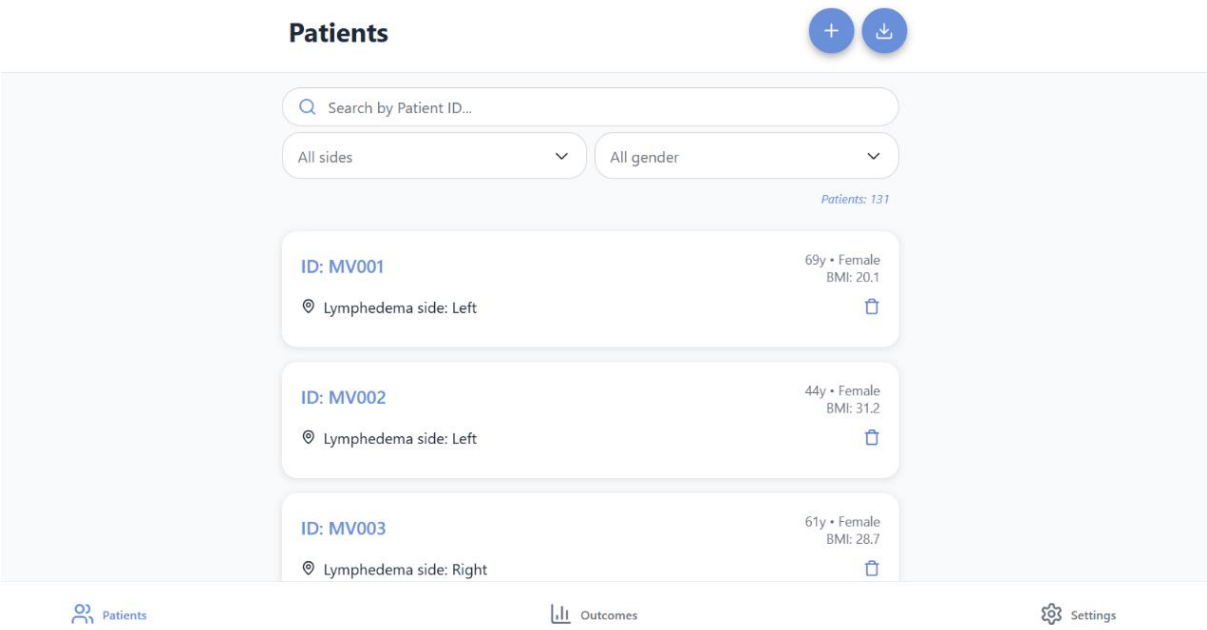
If the page loads correctly, the login screen appears:



The platform does **not** allow account creation. Since LymphTrack is a clinical data visualization tool, only predefined accounts created by an administrator can be used to sign in.

Enter your email address and password, then click **Sign In**.

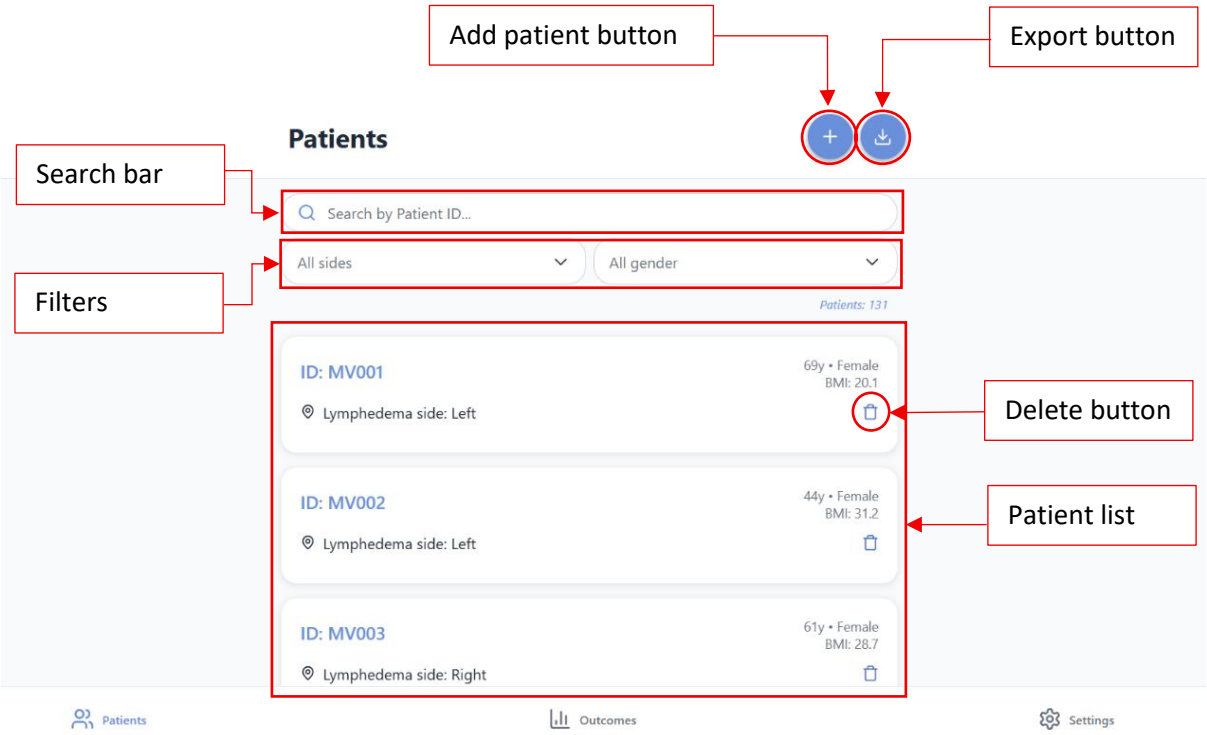
After logging in, you are redirected to the Patients page:



7.2 Patients page

Once connected, the home page of the application displays the **Patients** screen. This interface provides access to all registered patients and allows the user to create, search, filter, or export patient-related data.

Below is a screenshot of the Patients page with its main controls highlighted:



Overview of the interface

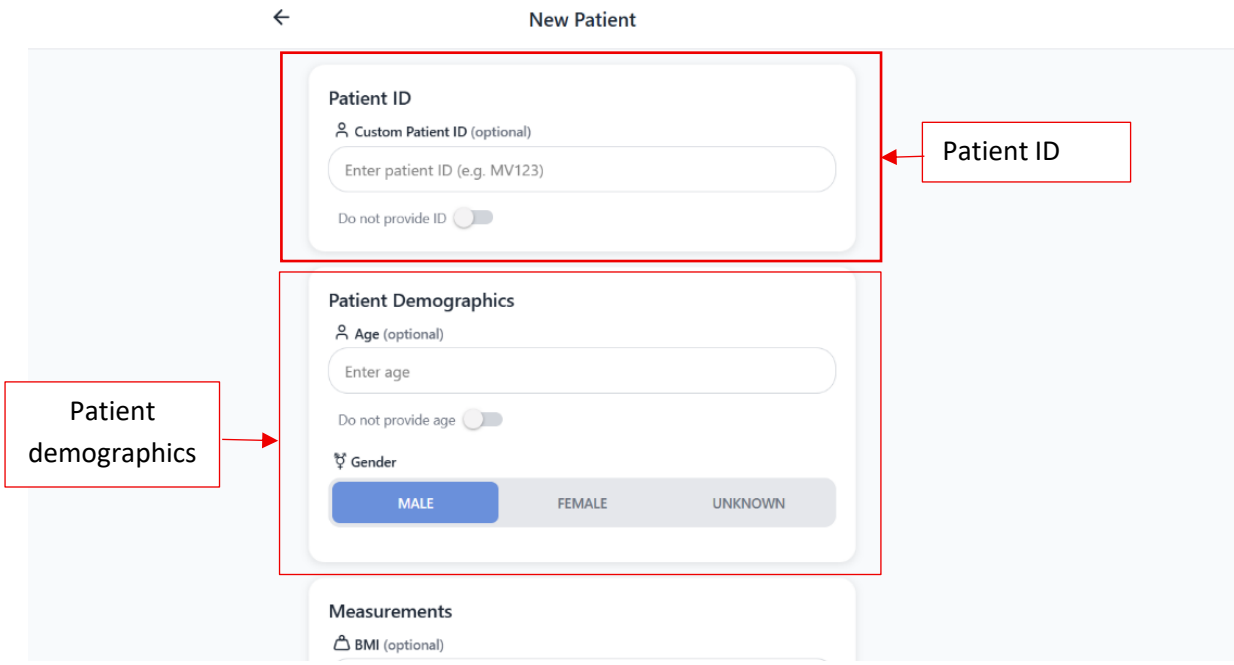
- **Search bar:** Search for a patient by ID (e.g., “MV005”).
- **Filters:** Filter patients by lymphedema side (left, right, bilateral, all) or gender (female, male, unknown).
- **Add patient button:** Opens the form used to register a new patient.
- **Export button:** Allows exporting one or multiple patient folders as a ZIP file.
- **Patient list:** Displays all registered patients along with their ID, age, gender, BMI, and lymphedema side.
- **Delete button:** Removes a patient and all associated data.

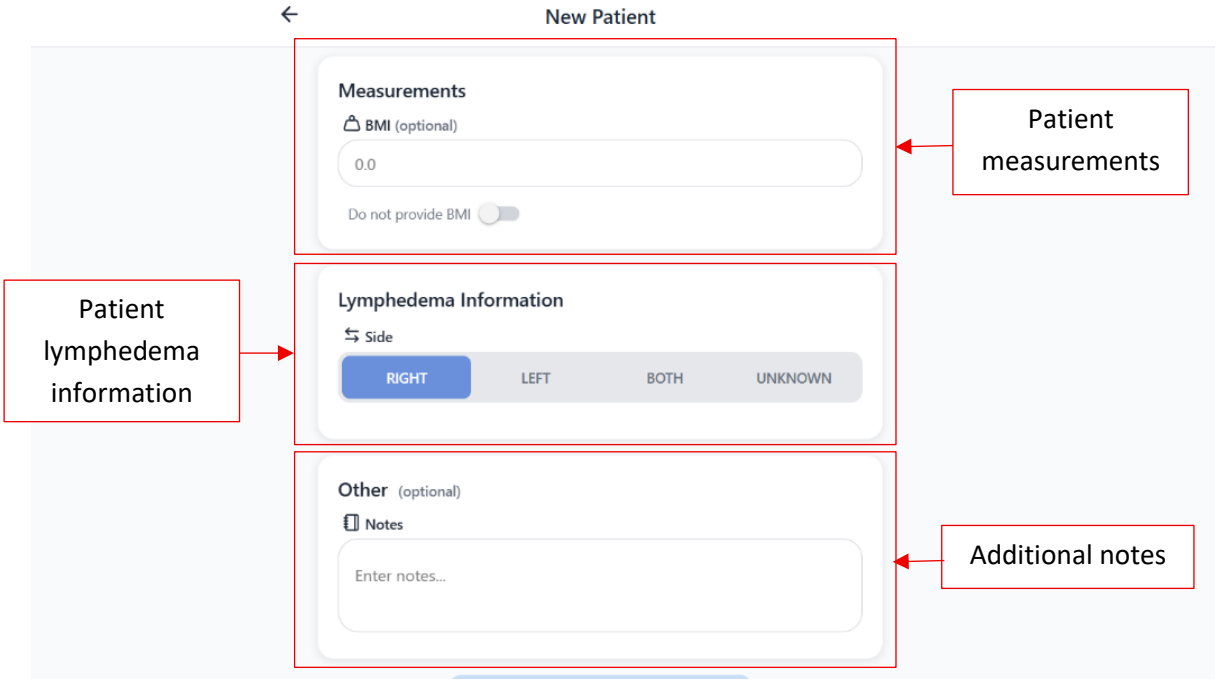
These elements are highlighted in the annotated screenshot below.

7.3 Creating a new patient

To create a new patient, click the **Add Patient (+)** button in the top-right corner of the Patients page.
You will be redirected to the **New Patient** form, where several optional or mandatory fields can be entered depending on the available clinical information.

Below is an annotated screenshot of the patient creation interface:





Overview of the interface

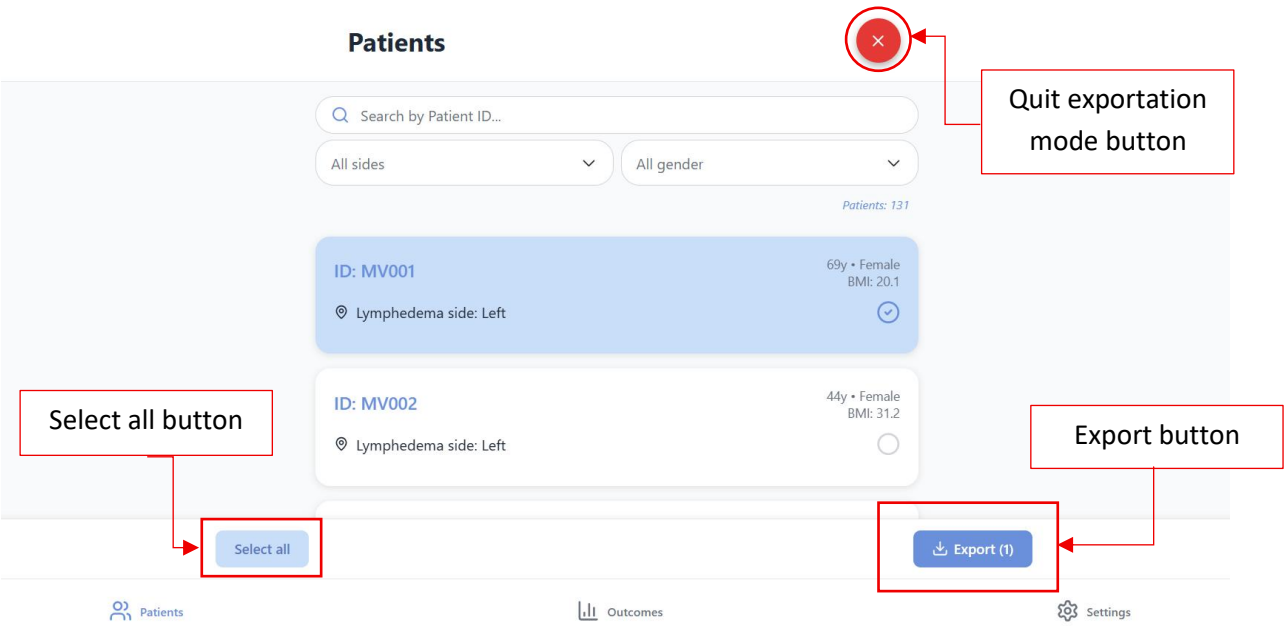
- **Patient ID:** You can enter a custom ID (e.g., *MV123*) or let the system generate one automatically using the toggle *"Do not provide ID"*. The ID uniquely identifies the patient.
- **Patient Demographics:** Optional fields such as age and gender (Male, Female, Unknown) help classify and filter patients.
- **Measurements:** An optional BMI value can be provided or skipped using the dedicated toggle.

- **Lymphedema Information:** Select the affected side: Right, Left, Both, or Unknown. This information appears in the patient list.
- **Notes (optional):** A free-text area is available for additional clinical information or observations.

These elements are highlighted in the annotated screenshot below.

7.4 Exporting patient data

To export patient data, click the **Export** button (download icon) in the top-right corner of the Patients page. This action enables a selection mode where check circles appear on each patient card, allowing you to choose which patient folders you want to download.



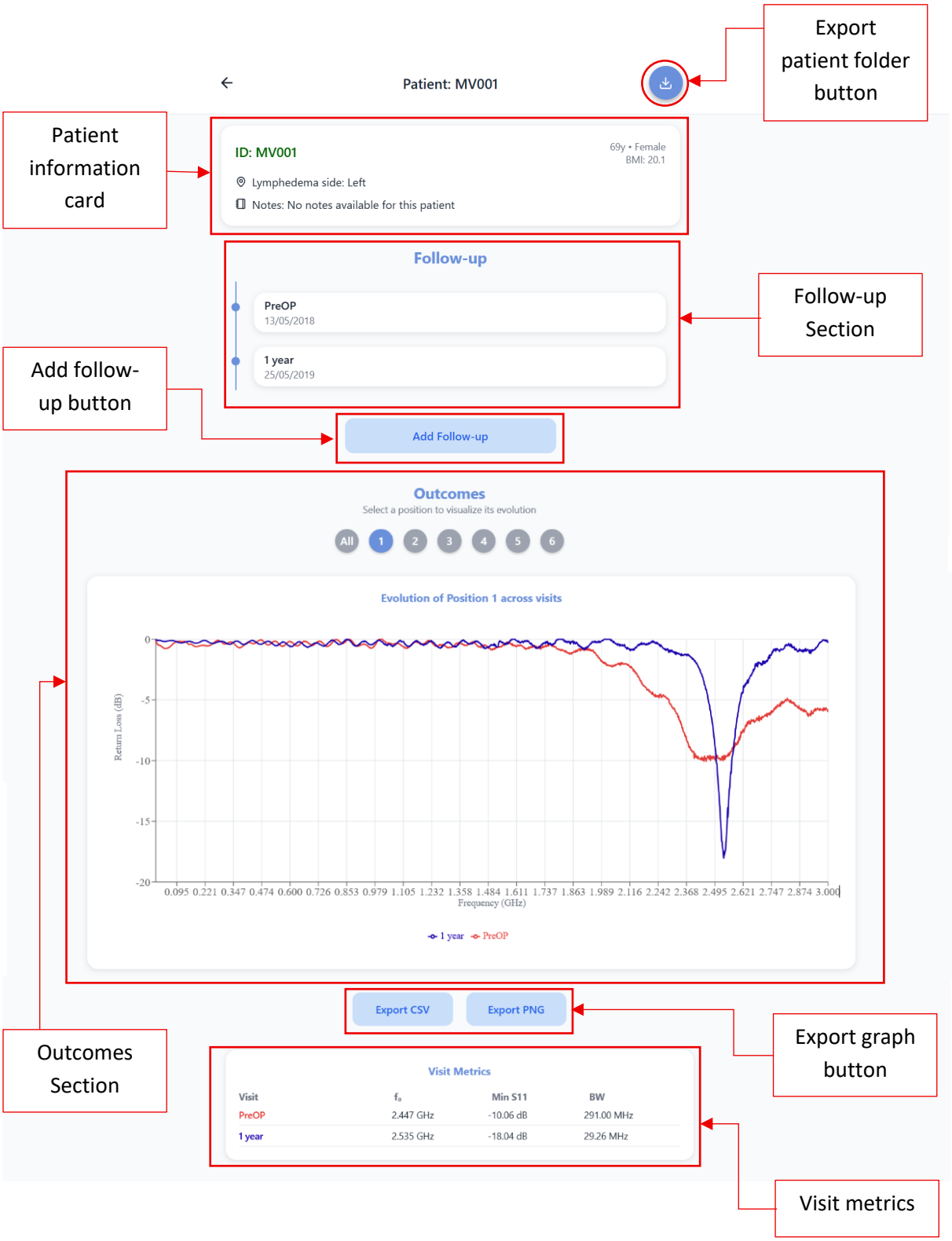
Overview of the interface

- **Select all button:** Selects every patient in the list for export.
- **Export button:** Downloads the selected patient folders as a ZIP file.
- **Quit exportation mode button:** Exits the selection mode and returns to the normal Patients view.

7.5 Viewing a patient profile

When you click on a patient in the Patients list, you are redirected to the Patient Page, which displays all the information related to that patient, including demographics, clinical notes, follow-up visits, and measurement visualizations.

Below is an example of the patient interface using patient **MV001**:



Overview of the interface

- **Patient information card:** Displays the patient's ID, age, gender, BMI, lymphedema side and clinical notes.
- **Export patient folder button:** Allows exporting the complete folder of this specific patient as a ZIP file, including all visits, measurements and photos.

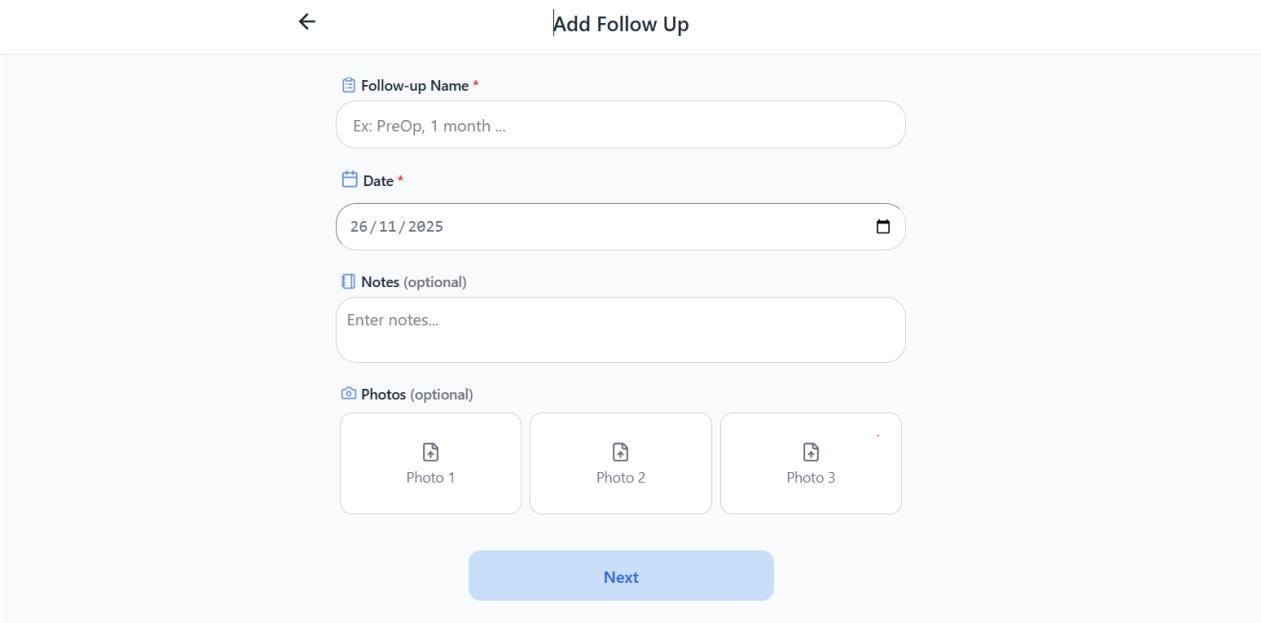
- **Follow-up section:** Lists all visits previously recorded for the patient, including the visit name and date.
- **Add Follow-up button:** Opens the form to register a new visit and import measurement files for this patient.
- **Outcomes section:** Allows selecting one of the six measurement positions (or "All") to visualize S11 evolution across visits.
- **Graph visualization:** Shows the S11 return loss curves for the selected position, with each visit represented in a different color.
- **Graph export buttons:** Provide options to download the displayed graph as a PNG image or export the raw measurement data as a CSV file.
- **Visit metrics table:** Summarizes key values extracted from each visit (resonant frequency f_0, minimum S11, bandwidth), enabling comparison between follow-ups.

These elements are illustrated in the annotated screenshot below.

7.6 Creating a follow-up visit

To create a new follow-up visit for a patient, click the **Add Follow-up** button on the patient page. You will be redirected to the Add Follow Up form, where you can enter the visit details (name, date, notes, photos) before importing the measurement files for each position.

Below is a screenshot of the follow-up creation interface:



Overview of the interface

- **Follow-up Name:** Mandatory field where you specify the name of the visit (e.g., *PreOP*, *1 month*, *1 year*).
- **Date:** Mandatory field indicating when the measurement session was performed.

- **Notes (optional):** A free-text field to store clinical observations or contextual information related to the visit.
- **Photos (optional):** Up to three photos can be added (e.g., clinical pictures taken during the visit).
- **Next button:** Once the information is entered, click Next to proceed to the measurement positions screen and upload the Excel/CSV files for this visit.

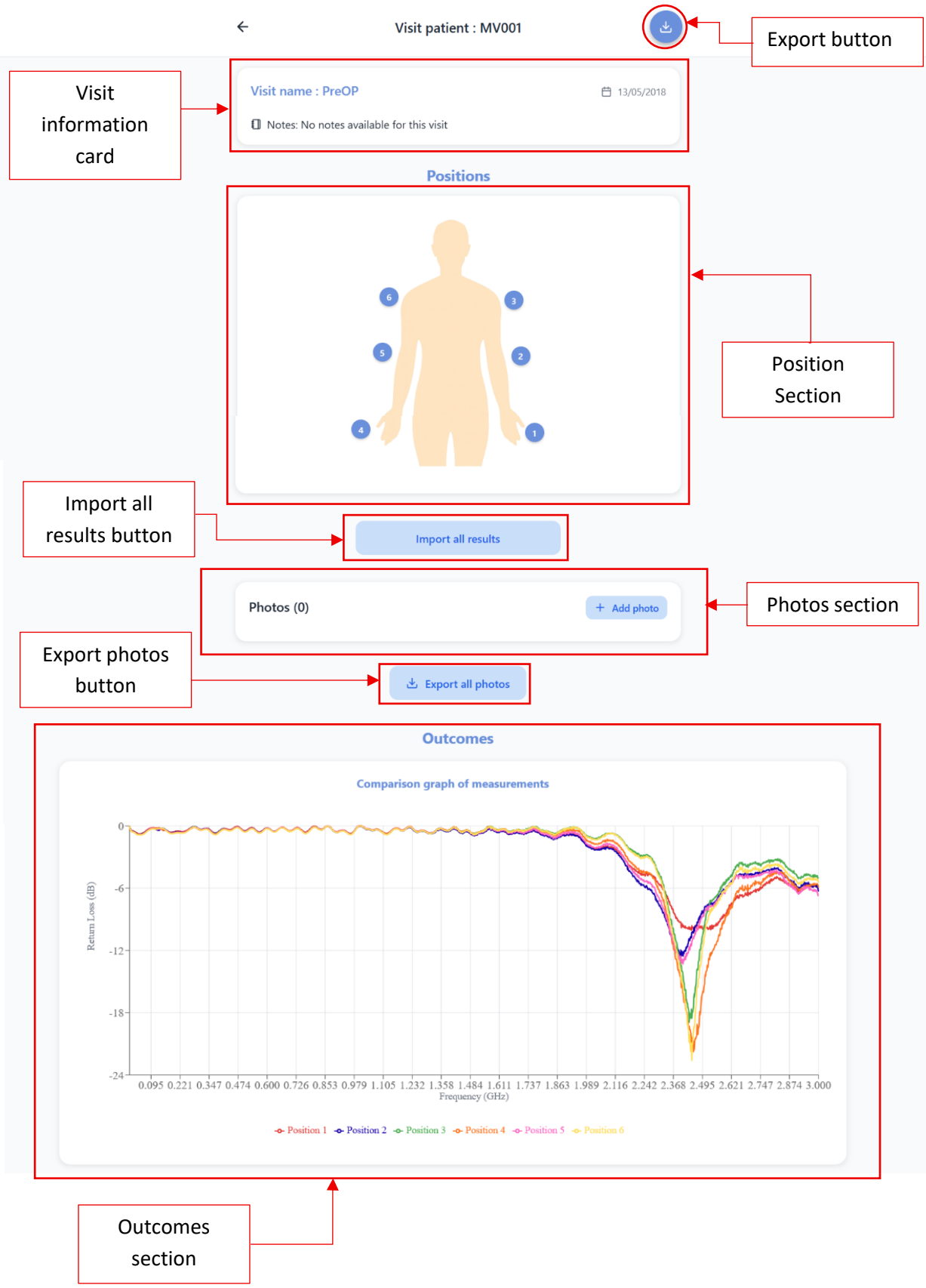
7.7 Viewing a patient visit

After creating a follow-up, the application redirects you to the Visit Page, where all information related to this specific visit is displayed.

Alternatively, you can access this page at any time by clicking on a visit card in the Follow-up section of the patient profile.

From this page, you can review visit details, upload measurement files for each position, add photos, and visualize all measurements recorded during the session.

Below is an annotated screenshot of the visit interface:


Visit patient : MV001
Export button
Visit information card
Visit name : PreOP
13/05/2018
Notes: No notes available for this visit
Positions
Position Section
Import all results button
Import all results
Photos (0)
Add photo
Photos section
Export photos button
Export all photos
Outcomes
Comparison graph of measurements
Return Loss (dB)
Frequency (GHz)
Position 1
Position 2
Position 3
Position 4
Position 5
Position 6
Outcomes section

Overview of the interface

- **Visit information card:** Displays the visit name, date and any notes associated with this follow-up session.
- **Export button:** Allows exporting the entire visit folder as a ZIP file, including all measurement files and photos.
- **Position section:** Shows the six measurement positions. Each numbered button (1–6) opens the dedicated page to upload or review the measurement files for that specific position.
- **Import all results button:** Lets you import all measurement files at once, but only if you have a complete dataset, meaning 18 Excel files (3 measurements × 6 positions). If fewer files are available, measurements must be uploaded position by position.
- **Photos section:** Allows adding up to three photos taken during the visit. These photos may help in documenting the clinical appearance or swelling evolution.
- **Export photos button:** Downloads all photos attached to this visit as a ZIP file.
- **Outcomes section:** Displays a comparison graph of measurements across all six positions for the selected visit. Each curve corresponds to one measurement position and is color-coded.

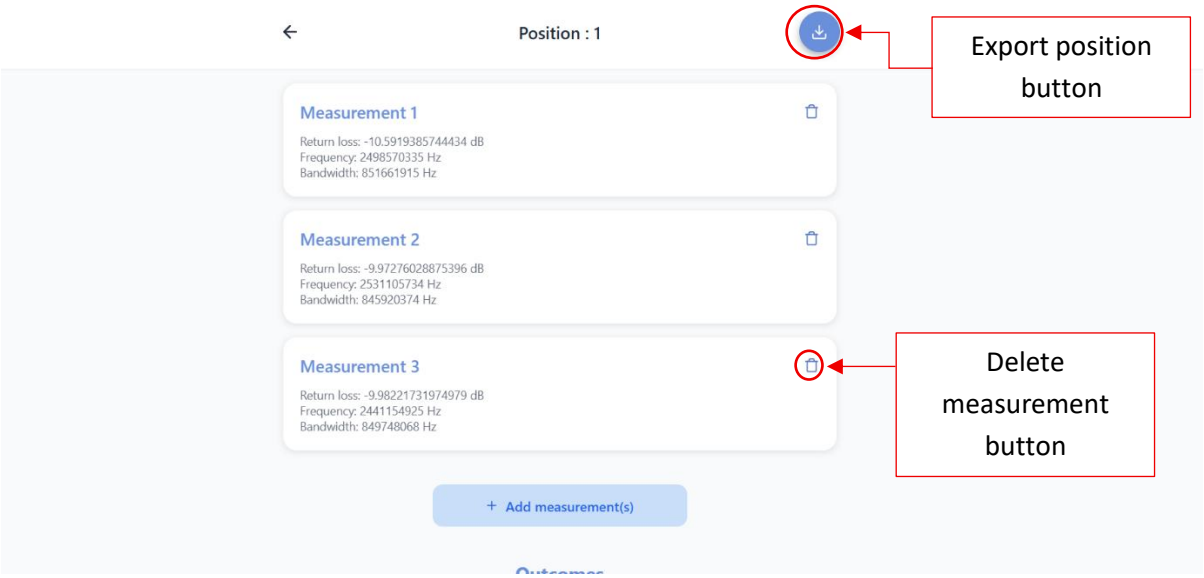
These elements are illustrated in the annotated screenshot above.

7.8 Position page

When selecting a position from the visit interface, the application opens the Position Page, where all measurements recorded for this specific position are listed.

From here, you can upload new measurement files, review existing ones, delete individual measurements, and visualize the comparison graph for the current visit.

Below is an annotated screenshot of the position interface:



Overview of the interface

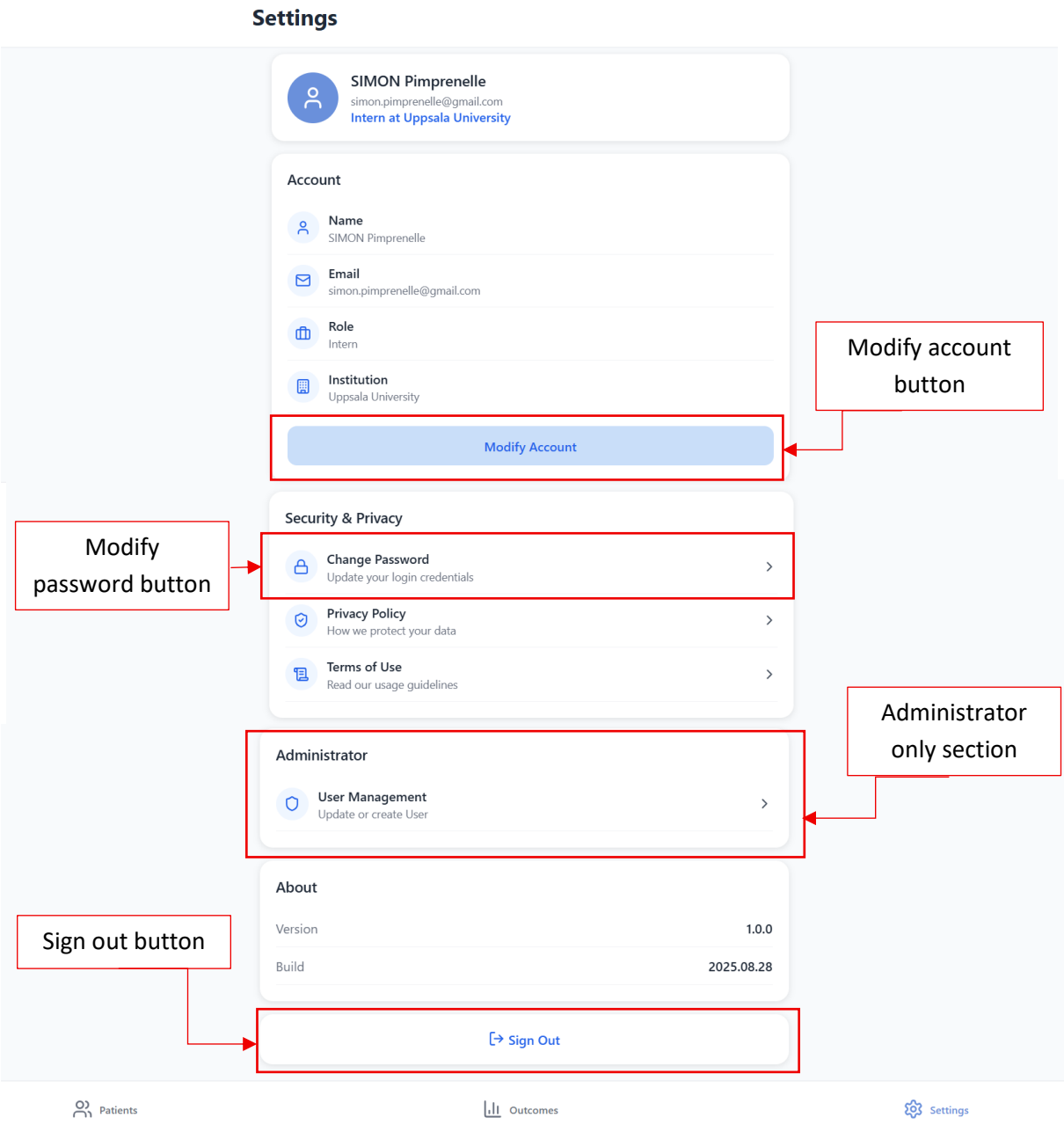
- **Comparison graph of measurements:** Displays all measurements imported for this position. Each curve is color-coded. The graph helps visualize measurement consistency and detect anomalies.
- **Delete measurement button:** Allows removing a specific measurement. If a measurement is deleted, the numbering automatically updates to remain consistent (e.g., Measurement 1, Measurement 2…).
- **Export position button:** Allows exporting all measurement files, graphs and processed data related to this specific position as a ZIP file

7.9 Settings

The *Settings* page provides access to all account-related options, including personal information, password management and administrator tools. It is divided into several sections:

- **Personal Account:** Displays the user's full name, email, role and institution. The *Modify Account* button allows updating this information.
- **Security & Privacy:** Contains options to change your password, view the privacy policy and access the application's terms of use.
- **Administrator Section (admin accounts only):** Provides access to user management tools, allowing administrators to create, edit or delete user accounts.
- **Sign Out:** Logs the user out of the application.

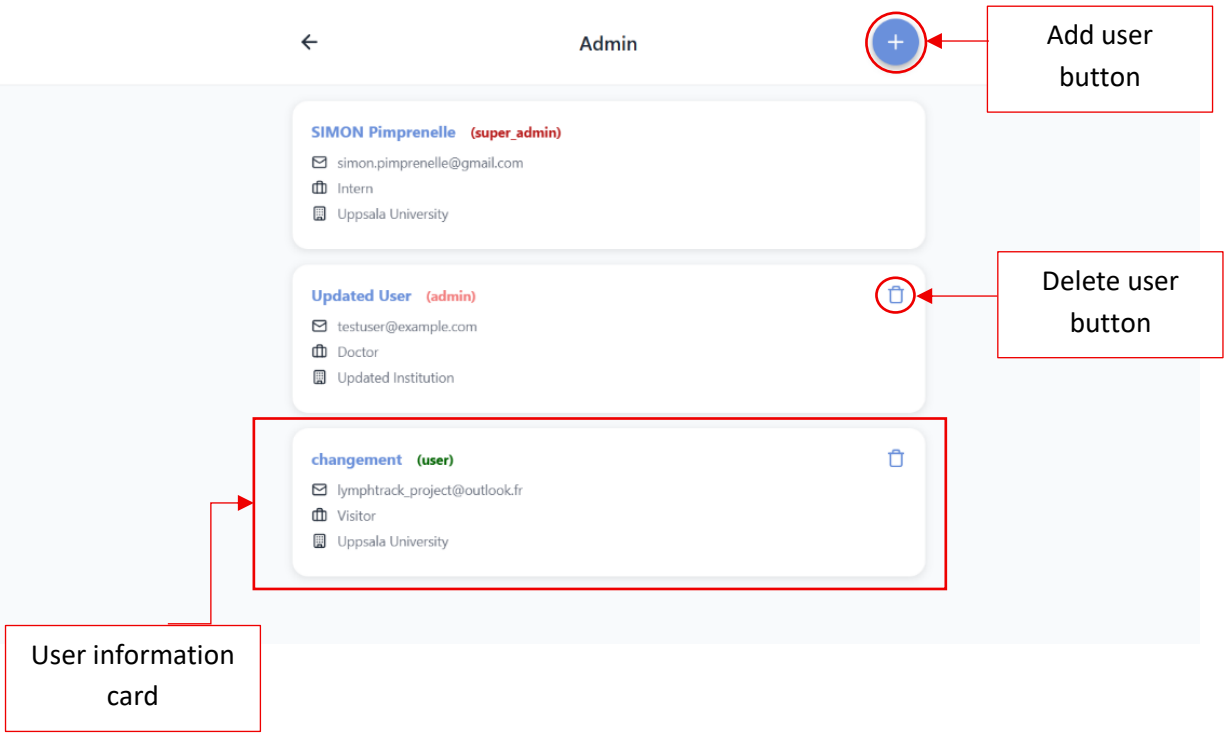
These elements are illustrated in the annotated screenshot below :



7.9.1 Administrator – User management

The *User Management* interface is accessible only to administrators. From this page, administrators can view all registered users, create new accounts, or delete existing ones.

Below is an annotated screenshot of the user management interface:



Overview of the interface

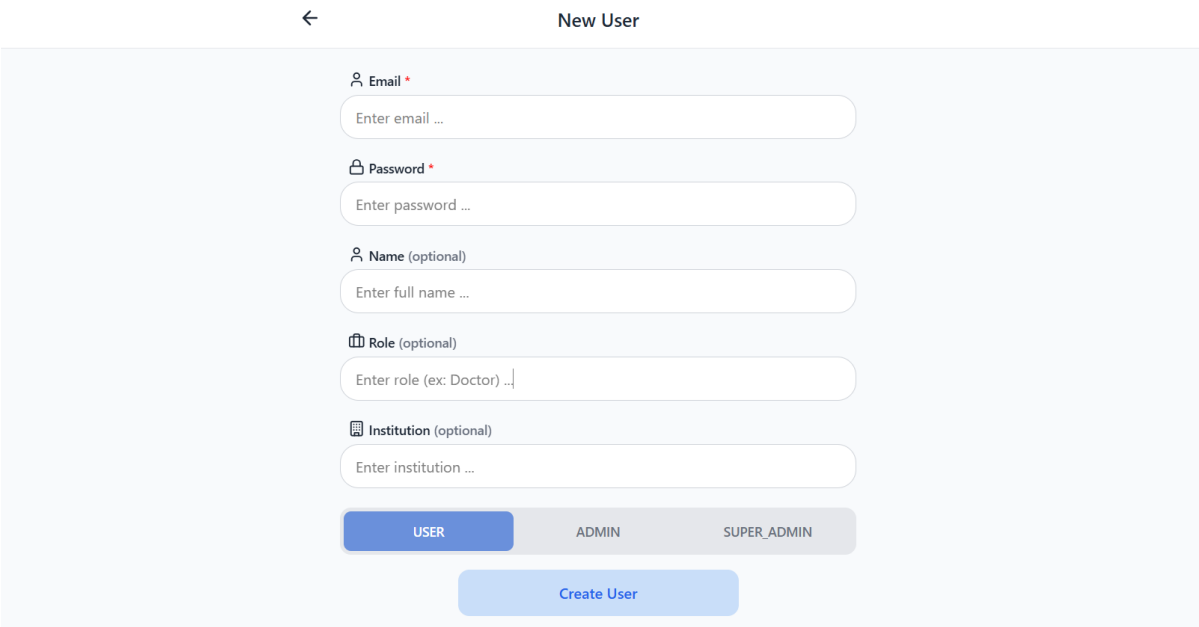
- **User information card:** Shows the user's name, email, role (user, admin or super_admin) and institution. The role determines the level of access within the application.
- **Add user button:** Allows administrators to create a new user account by entering personal details and selecting a role.
- **Delete user button:** Deletes the selected account.
- **Admin self-protection:** For security reasons, an administrator cannot delete their own account. This prevents accidental loss of admin access.

7.9.2 Administrator – User creation and update

Administrators can create and manage user accounts from the *User Management* section.

This interface allows adding new users, assigning roles and editing existing accounts. The form used to modify a user is identical to the one used to create a new account.

Below is a screenshot of the user creation interface:



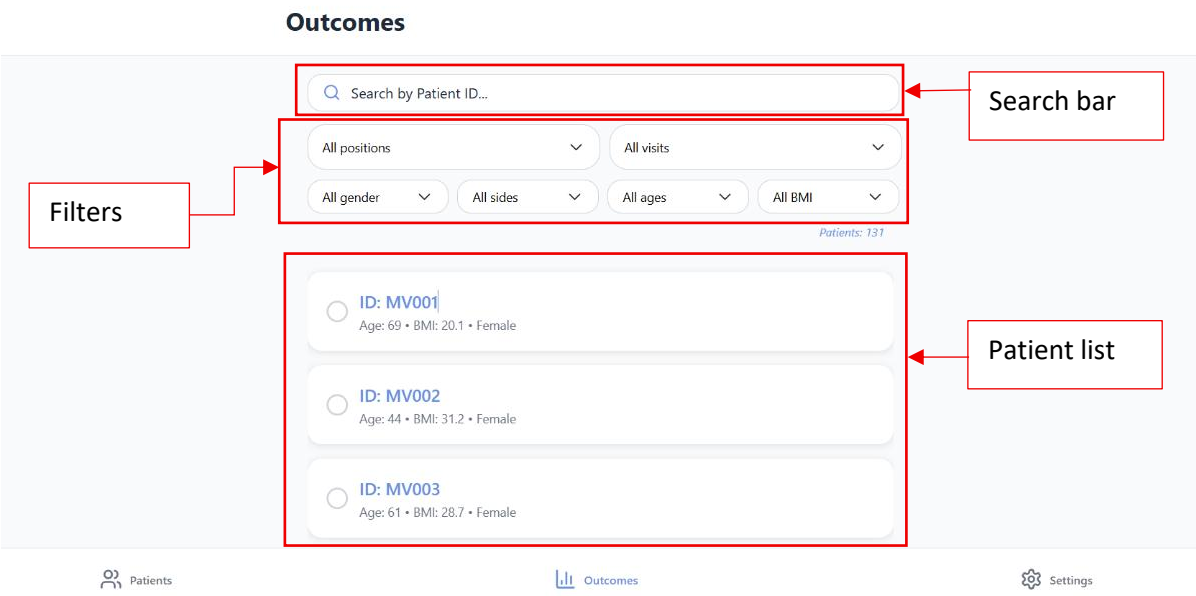
Overview of the interface

- **Email (required):** The email address used for login.
- **Password (required):** The password assigned to the new user.
- **Name (optional):** The user's full name.
- **Role (optional):** Additional information describing the user's position (e.g., “Doctor”, “Nurse”, “Physiotherapist”).
- **Institution (optional):** The organization or institution the user belongs to.
- **Access Level selector:** Lets the administrator choose the user’s access level:
    - **USER:** Standard account with read-only or limited interaction permissions.
    - **ADMIN:** Can create and edit users, but cannot edit super_admin accounts.
    - **SUPER_ADMIN:** Full permissions, including managing other admins and super_admins.
- **Create User button:** Finalizes the creation of the new account.

7.10 Outcomes

The *Outcomes* page allows comparing microwave measurements across multiple patients. This interface provides several filters to select which patients, visits, positions or demographic criteria should be included in the analysis.

Below is the main Outcomes interface, with its filters:



Overview of the interface

- **Search bar:** Allows searching for patients by ID.
- **Visit filter:** Select a specific visit (e.g., *PreOP*, *1 month*, *1 year*) or view *All visits*.
- **Position filter:** Choose one of the six measurement positions or select *All positions*.
- **Gender filter:** Filter patients by gender (Male, Female, all).
- **Lymphedema side filter:** Filter by affected side (Left, Right, Bilateral, All).
- **Age filter:** Restrict the selection to an age range.
- **BMI filter:** Restrict the selection to a BMI range.
- **Patient selection:** Use the round selection button to include patients in the comparison.
- **Show results button:** Displays the comparison graphs based on the selected parameters.

These elements allow clinicians to build precise comparisons between patients.

Three types of comparisons possible

The Outcomes module supports **three different comparison modes**, depending on the chosen filters:

1. **Compare *positions* across multiple patients (Position-based comparison)**

Used when:

- Position is selected (1–6)
- Visit = *All visits*

For each patient, the S11 evolution for *Position 1* (for example) across all their visits will be drawn. You compare **how the same position evolves over time for each patient**.

2. **Compare *visits* across multiple patients (Visit-based comparison)**

Used when:

- A visit is selected (e.g., *1 month*)
- Position = *All positions*

The system generates **six graphs** (one per position). For each graph, the curves of all selected patients are displayed.

3. **Compare *a visit AND a position* across multiple patients (Cross-comparison)**

Used when:

- A visit is selected (e.g., *1 month*)
- A position is selected (e.g., *Position 1*)

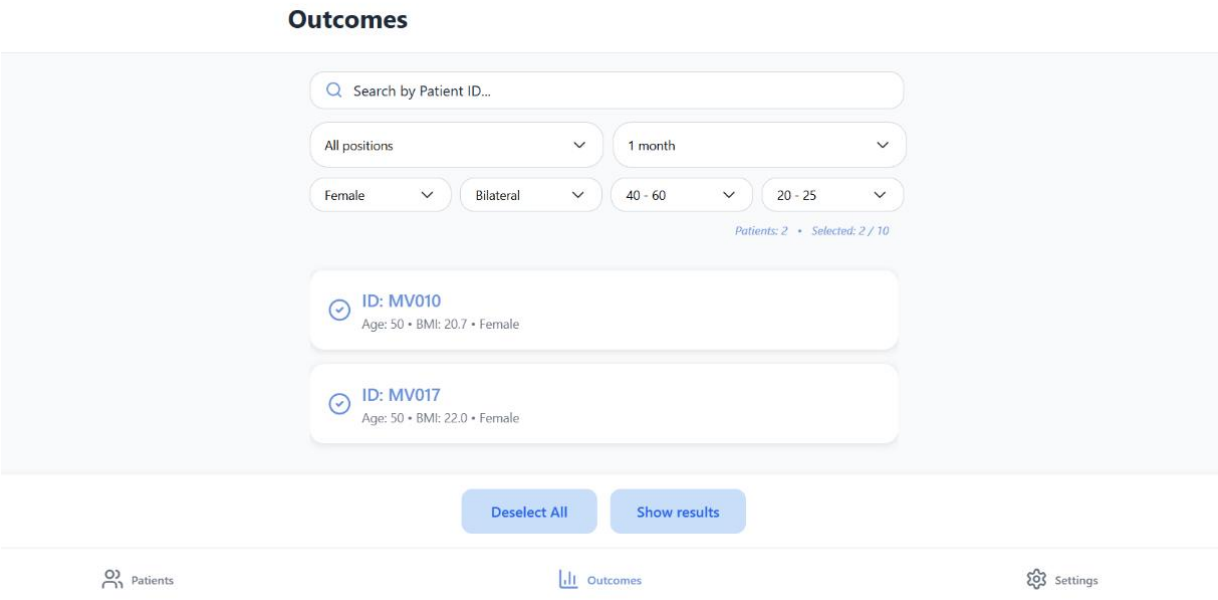
For several patients, you compare the curves recorded at the same visit and same position.

Example – Visit-based comparison

Here is an example using two patients (**MV010** and **MV017**) and the visit **1 month**, with *All positions* selected.

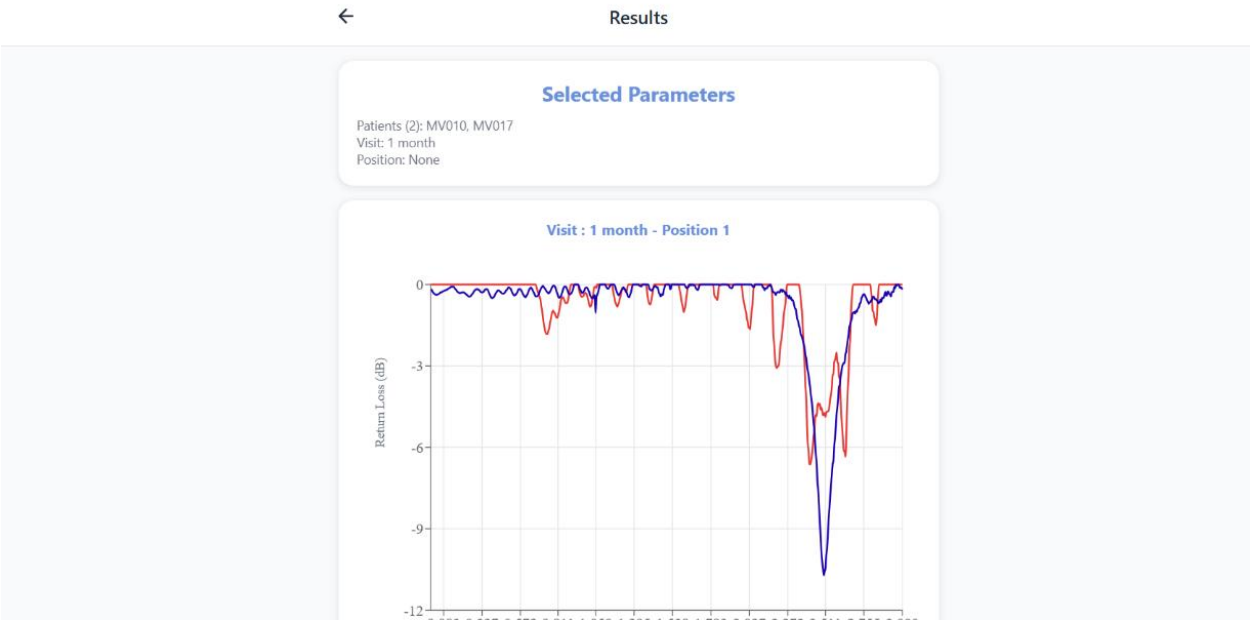
After selecting:

- Visit: **1 month**
- Position: **All positions**
- Patients: **MV010**, **MV017**



When clicking on **Show results, t**he application displays a comparison graph for each of the six positions.

Below is an example for **Position 1**, comparing the two patients during the same visit:



This mode allows evaluating differences between patients during the same visit across all positions.

8. Future improvements and known issues

This section highlights the main improvements that could be implemented in future iterations of LymphTrack, as well as the current limitations that remain outside the project's scope.

It serves as a roadmap for future contributors.

8.1 Future improvements

1. Deploy the backend and frontend using a stable production setup

The application currently runs on a university remote machine in a local HTTP environment.

Future work could include:

- hosting the backend behind a reverse proxy (e.g., Nginx, Traefik),
- enabling HTTPS with a domain and TLS certificate,
- converting the current local setup into a production-ready deployment (Docker, CI/CD).

This would make the system easier to access and maintain long-term.

2. Extend the measurement processing pipeline

Current processing is limited to:

- Min S11
- Resonance frequency
- Bandwidth

Future improvements could include:

- Feature extraction across the full curve
- Automatic anomaly detection
- Curve smoothing / normalization
- Multi-session comparison heatmaps
- Integration with machine learning models

3. Expand the healthy patient module

The healthy_patients and healthy_metadata tables are implemented but currently unused. Future work could include:

- Healthy vs. affected asymmetry ratios
- Automatic threshold computation
- Statistical baselines per position
- Longitudinal comparisons with healthy cohorts

This would strengthen the clinical relevance of the application.

4. Implement full role-based dashboards

Roles (user, admin, super_admin) are implemented, but the frontend UI displays the same layout for all. Possible improvements:

- Admin dashboard for user management
- Super admin panel for advanced operations
- Limited “read-only mode” UI for clinicians

5. Better error handling and UI messaging

Some frontend actions currently fail silently (e.g., failed API calls). Improvements can include:

- Toast notifications for errors
- Loading states for slow operations
- Clear feedback after uploads or failures
- Improved mobile responsiveness

6. Display user responsible for creating or editing entries

Although the database already stores the user who created or updated a patient, visit or measurement result (via created_by and updated_by fields), this information is currently not shown in the interface.

A future enhancement would be to display this metadata in the UI, at least for **admin** and **super_admin** accounts. This would improve traceability, accountability and auditability in clinical environments.

7. Restrict patient folder export permissions

At the moment, **any user** can export a patient's complete folder (including visits, measurements and photos). In a future version, export permissions could be restricted so that only **admin** and **super_admin** roles are allowed to download patient folders. This would strengthen data protection and enforce stricter access control policies.

8.2 Known issues

Here are the issues that were identified but could not be fully fixed due to time constraints or dependency limitations.

1. File structure sensitivity

Some Excel files require manual renaming or reformatting before processing. Unexpected column names or missing values may cause parsing errors.

2. Limited user-input validation

Some frontend forms still accept incomplete or invalid values. Future improvements should enforce stricter validation rules.

3. Web version technical constraints

The Expo Web build is functional but subject to limitations of React Native Web, especially regarding navigation and gesture handling. Certain animations or touch behaviours may behave differently across browsers.

9. Annex

9.1 Annex 1: LymphTrackApp directory structure

This annex provides the complete directory structure of the LymphTrack project, including backend, frontend, database schema and all related configuration files.

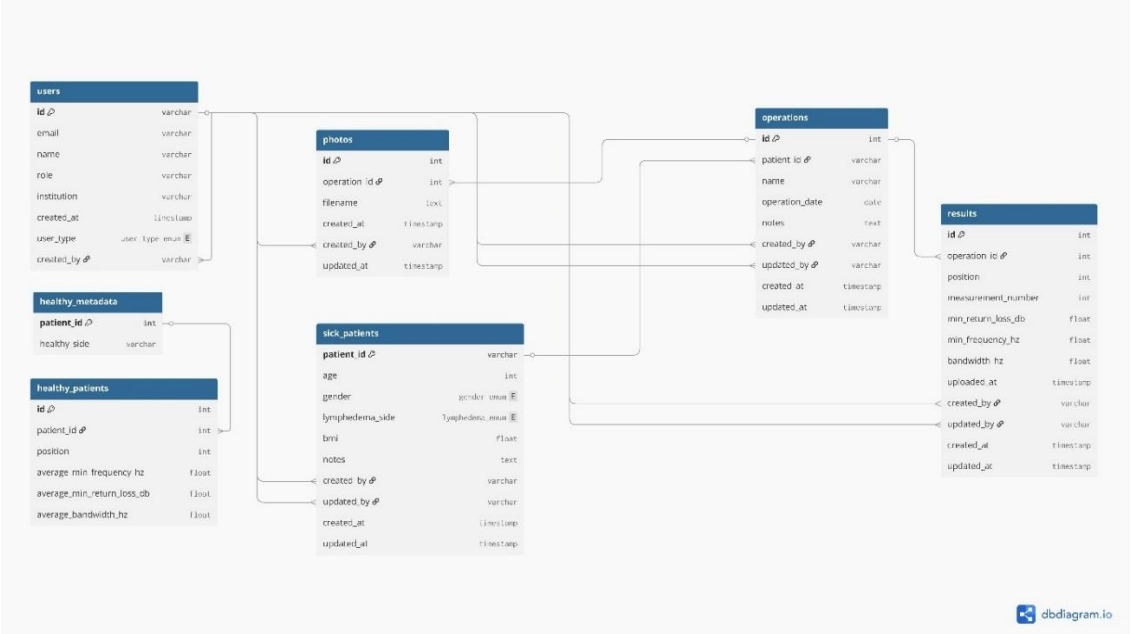
```
C:\LymphTrackApp
|
|    .gitignore
|    docker-compose.yml
|    README.mg
|    start_app.ps1
|
+---.expo/
|
+---.vscode/
|
+---database/
|          schema.sql       // Database schema defining all PostgreSQL tables
|
+---backend/
|     |    .env
|     |    requirements.txt  // Python dependencies required by the FastAPI backend
|     |
|     +--- app/                     // Main FastAPI application package (API, DB, config)
|     |     |   main.py
|     |     |
|     |     +---api/        // REST API endpoints consumed by the frontend
|     |     |     operations.py
|     |     |     patients.py
|     |     |     photos.py
|     |     |     results.py
|     |     |     users.py
|     |     |
|     |     +---auth/       // Authentication helpers and JWT dependencies
|     |     |     deps.py
|     |     |
|     |     +---core/       // Application configuration and environment settings
|     |     |     config.py
|     |     |
|     |     +---db/         // Database connection + ORM models (SQLAlchemy)
|     |     |     database.py
|     |     |     models.py
|     |     |
|     |     \---tests/              // Automated backend tests (pytest)
|     |           |   test.jpg
|     |           |   test2.jpg
|     |           |   VNA_test.xls
|     |           |
|     |           +---api/          // Tests for API endpoints
|     |           |     test_operations.py
|     |           |     test_patients.py
|     |           |     test_photos.py
|     |           |     test_results.py
|     |           |     test_users.py
|     |           |
|     |           \---db/           // Tests for database connectivity and models
|     |                 test_db.py
|     |
|     \--- venv/
|
```

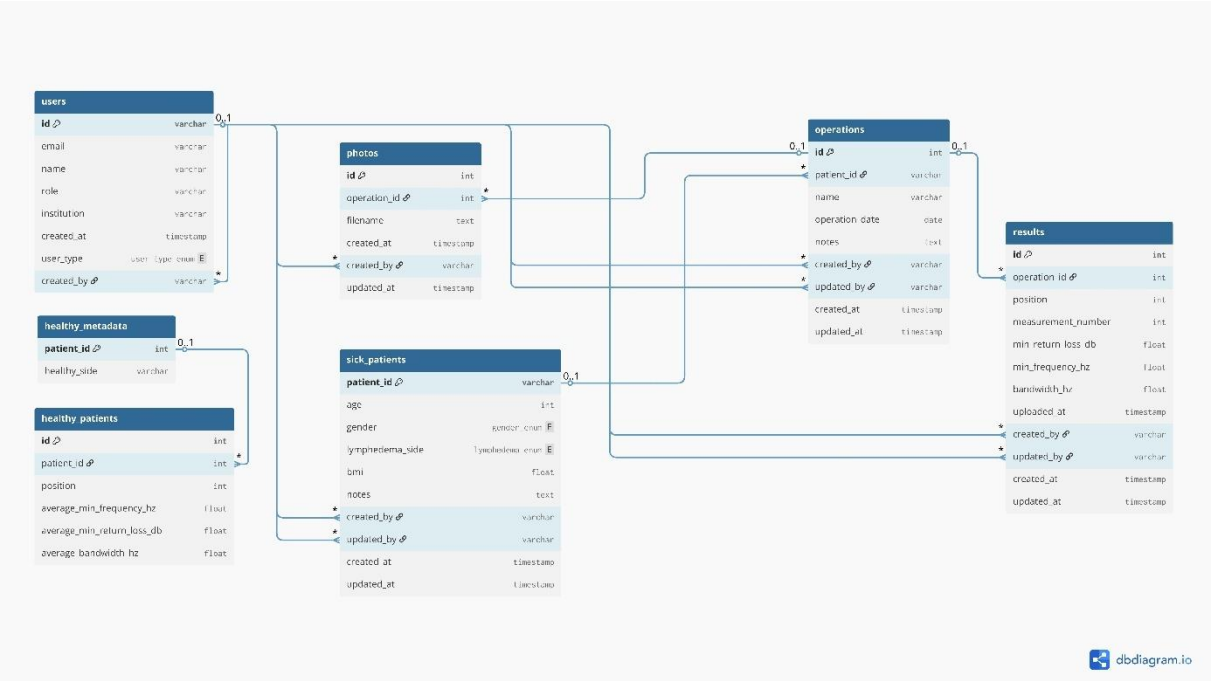
```
\---frontend/
      |    app.json
      |    package.json
      |    .gitignore
      |    tsconfig.json
      |
      +---.expo/     // Expo runtime metadata and development artifacts
      |
      +---.next/     // Production build output (generated by Next.js)
      |
      +---assets/    // Static frontend assets (icons, images)
      |     |
      |     +---icons/
      |     \---images/
      |
      +---dist/      // Compiled frontend (web version) used for deployment
      |
      +---components/        // Reusable UI components used across screens
      |         filterDropDown.tsx
      |         graphView.tsx
      |         inputField.tsx
      |         loadingScree.tsx
      |         SegmentedControl.tsx
      |
      +---constants/         // Global constants (API URL, colors, global styles)
      |         api.ts
      |         colors.ts
      |         styles.ts
      |
      +---hooks/
      |         useFrameworkReady.ts
      |
      +---lib/               // Supabase client (auth + storage integration)
      |         supabase.ts
      |
      +---node_modules/             // External dependencies installed via npm
      |
      +---utils/
      |         alertUtils.ts
      |         dateUtils.ts
      |         deleteUtils.ts
      |         exportGraphUtils.ts
      |         exportUtils.ts
      |         patientUtils.ts
      |         uploadUtils.ts
      |         visitUtils.ts
      |         api.ts
      |
      \---app/
            |   index.tsx
            |   +not-found.tsx
            |   _layout.tsx
            |
            +---(auth)/
            |       login.tsx
            |
            +---(tabs)/
            |       _layout.tsx
            |       outcomes.tsx
            |       patients.tsx
            |       settings.tsx
            |
            +---outcomes/
            |       outcomesResults.tsx
            |
            +---patient/
```

```
            |       |   create.tsx
            |       |   [patient_id].tsx
            |       |
            |      +---followup/
            |       |       |   [id_operation].tsx
            |       |       |
            |       |       +---create_followup/
            |       |       |       [patient_id].tsx
            |       |       |
            |       |       +---modify_followup/
            |       |       |       [id_operation].tsx
            |       |       |
            |       |       \---position/
            |       |               [position].tsx
            |       |
            |      \---modify/
            |               [patient_id].tsx
            |
            \---setting/
                    |
                    +---account/
                    |       modify_account.tsx
                    |
                    +---admin/
                    |       |   admin.tsx
                    |       |
                    |       \---user/
                    |               create_user.tsx
                    |               [user_id].tsx
                    |
                    \---security/
                            modify_password.tsx
                            privacy_policy.tsx
                            terms_of_use.tsx
```

9.2 Annex 2: Database schema diagram

This annex contains the full relational database diagram used by LymphTrack. It illustrates all tables, relationships and constraints implemented in the Supabase PostgreSQL database.





9.3 Annex 3: SQL schema

This annex provides the complete SQL schema used to create the database tables. It includes all table definitions, constraints, enumerations, and relationships.

```
-- WARNING: This schema is for context only and is not meant to be run.
-- Table order and constraints may not be valid for execution.

CREATE TABLE public.healthy_metadata (
  patient_id integer NOT NULL UNIQUE,
  healthy_side character varying NOT NULL,
  CONSTRAINT healthy_metadata_pkey PRIMARY KEY (patient_id)
);
CREATE TABLE public.healthy_patients (
  id integer NOT NULL DEFAULT nextval('healthy_patients_id_seq'::regclass),
  patient_id integer NOT NULL,
  position integer NOT NULL CHECK ("position" >= 1 AND "position" <= 6),
  average_min_frequency_hz double precision,
  average_min_return_loss_db double precision,
  average_bandwidth_hz double precision,
  CONSTRAINT healthy_patients_pkey PRIMARY KEY (id),
  CONSTRAINT healthy_patients_patient_id_fkey FOREIGN KEY (patient_id) REFERENCES
public.healthy_metadata(patient_id)
);
CREATE TABLE public.operations (
  id integer NOT NULL DEFAULT nextval('operations_id_operation_seq'::regclass),
  patient_id character varying NOT NULL,
  name character varying NOT NULL,
  operation_date date NOT NULL,
  notes text,
  created_by character varying,
  updated_by character varying,
  created_at timestamp without time zone DEFAULT now(),
```

```
  updated_at timestamp without time zone DEFAULT now(),
  CONSTRAINT operations_pkey PRIMARY KEY (id),
  CONSTRAINT    operations_created_by_fkey    FOREIGN    KEY    (created_by)    REFERENCES
public.users(id),
  CONSTRAINT    operations_updated_by_fkey    FOREIGN    KEY    (updated_by)    REFERENCES
public.users(id),
  CONSTRAINT       fk_patient       FOREIGN       KEY       (patient_id)       REFERENCES
public.sick_patients(patient_id)
);
CREATE TABLE public.photos (
  id integer NOT NULL DEFAULT nextval('photos_id_seq'::regclass),
  operation_id integer,
  created_at timestamp without time zone DEFAULT now(),
  filename text NOT NULL DEFAULT ''::text,
  created_by character varying,
  updated_at timestamp without time zone DEFAULT now(),
  CONSTRAINT photos_pkey PRIMARY KEY (id),
  CONSTRAINT photos_created_by_fkey FOREIGN KEY (created_by) REFERENCES public.users(id),
  CONSTRAINT    photos_operation_id_fkey    FOREIGN    KEY    (operation_id)    REFERENCES
public.operations(id)
);
CREATE TABLE public.results (
  id integer NOT NULL DEFAULT nextval('results_id_seq'::regclass),
  operation_id integer NOT NULL,
  position integer CHECK ("position" >= 1 AND "position" <= 6),
  measurement_number integer,
  min_return_loss_db double precision,
  min_frequency_hz double precision,
  bandwidth_hz double precision,
  uploaded_at timestamp without time zone DEFAULT now(),
  created_by character varying,
  updated_by character varying,
  created_at timestamp without time zone DEFAULT now(),
  updated_at timestamp without time zone DEFAULT now(),
  CONSTRAINT results_pkey PRIMARY KEY (id),
  CONSTRAINT results_created_by_fkey FOREIGN KEY (created_by) REFERENCES public.users(id),
  CONSTRAINT results_updated_by_fkey FOREIGN KEY (updated_by) REFERENCES public.users(id),
  CONSTRAINT    results_operation_id_fkey    FOREIGN    KEY    (operation_id)    REFERENCES
public.operations(id)
);
CREATE TABLE public.sick_patients (
  patient_id character varying NOT NULL,
  age integer,
  gender USER-DEFINED,
  lymphedema_side USER-DEFINED,
  bmi double precision,
  notes text,
  created_by character varying,
  updated_by character varying,
  created_at timestamp without time zone DEFAULT now(),
  updated_at timestamp without time zone DEFAULT now(),
  CONSTRAINT sick_patients_pkey PRIMARY KEY (patient_id),
  CONSTRAINT   sick_patients_created_by_fkey   FOREIGN   KEY   (created_by)   REFERENCES
public.users(id),
  CONSTRAINT   sick_patients_updated_by_fkey   FOREIGN   KEY   (updated_by)   REFERENCES
public.users(id)
);
```

```
CREATE TABLE public.users (
  id character varying NOT NULL,
  email character varying NOT NULL,
  name character varying,
  role character varying,
  created_at timestamp without time zone,
  institution character varying,
  user_type USER-DEFINED NOT NULL DEFAULT 'user'::user_type_enum,
  created_by character varying,
  CONSTRAINT users_pkey PRIMARY KEY (id),
  CONSTRAINT users_created_by_fkey FOREIGN KEY (created_by) REFERENCES public.users(id)
);
```